Case Study - Employee Analytics System

Master Tables

```
create database employeeAnalytics;

use employeeAnalytics;

create table Departments(
dept_id int primary key,
dept_name varchar(50) not null unique
);

create table Locations(
location_id int primary key,
locatio_name varchar(50) not null unique
);

create table Job_Roles(
role_id int primary key,
role_name varchar(50) not null,
min_salary decimal(10,2),
max_salary decimal(10,2)
);
```

Employee Table

```
create table employees(
emp_id int primary key,
emp_name varchar(100) not null,
gender varchar(10),
dept_id int,
role_id int,
location_id int,
join_date date,
status varchar(20) default 'Active',
Foreign key(dept_id) references Departments(dept_id),
Foreign key(role_id) references Job_Roles(role_id),
Foreign key(location_id) references Locations(location_id)
);
```

Salary table

```
create table Salaries(
salary_id int primary key,
emp_id int,
salary decimal(10,2),
effective_date date,
foreign key(emp_id) references employees(emp_id)
);
```

Performance Table

```
create table Performance_Reviews(
review_id int primary key,
emp_id int,
review_year int,
rating decimal(2,1) check(rating between 1 and 5),
foreign key(emp_id) references employees(emp_id)
);
```

```
Insert into Departments values(
(1,’IT’),
(2,’HR’),
(3,’Finance’),
(4,’Marketing’),
(5,’Operations’));
```

Importing data

```
BULK INSERT Departments
FROM 'D:\Departments.csv'
WITH (
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n',
   TABLOCK
);
BULK INSERT Locations
FROM 'D:\Locations.csv'
WITH (
```

```
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    TABLOCK
);
BULK INSERT Job_Roles
FROM 'D:\job_roles.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    TABLOCK
);
BULK INSERT employees
FROM 'D:\employees.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    TABLOCK
);
BULK INSERT Salaries
FROM 'D:\salaries.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    TABLOCK
);
BULK INSERT Performance_reviews
FROM 'D:\Performance_reviews.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    TABLOCK
);
```

Performance Analytics

1.Top 10 high performing employees

This query calculates the average performance rating for each employee and then selects the top 10 employees with the highest average ratings.

Query

```
select top 10 e.emp_id ,e.emp_name,avg(pr.rating) as Average_Rating
from employees e join Performance_Reviews pr on
e.emp_id = pr.emp_id
group by e.emp_id,e.emp_name
order by avg(pr.rating) desc;
```

100 %

Results | Messages

	emp_id	emp_name	Average_Rating
1	111	Rohit Gupta	4.900000
2	143	Ashish Kapoor	4.900000
3	102	Priya Nair	4.800000
4	133	Yash Thakur	4.800000
5	128	Riya Sharma	4.700000
6	107	Vikram Rao	4.700000
7	141	Varun Nair	4.700000
8	138	Bhavna Joshi	4.600000
9	149	Prakash Nai...	4.600000
10	113	Manish Kumar	4.600000

2.Average rating per department

This query calculates the average performance rating of active employees in each department. It then lists all departments sorted from the highest to lowest average rating.

Query

```
select d.dept_id,d.dept_name,round(avg(pr.rating),2) as Average_Rating from Departments d
join employees e
on e.dept_id=d.dept_id
join Performance_Reviews pr
on e.emp_id=pr.emp_id
```

```
where e.status='Active'
group by d.dept_id,d.dept_name
order by avg(pr.rating) desc;
```

100 %

Results | Messages

	dept_id	dept_name	Average_Rating
1	1	IT	4.510000
2	3	Finance	4.410000
3	4	Marketing	4.410000
4	5	Operations	4.340000
5	2	HR	4.180000

3.Employees eligible for promotion(rating>4 & 2+ years)

This query retrieve data of employees who is eligible for promotion using the conditions that their performance rating is more than 4 and had 2 + years of experience

Query

```
select e.emp_id,e.emp_name,pr.rating from employees e
join Performance_Reviews pr
on e.emp_id =pr.emp_id
where pr.rating>4 and
datediff(year, e.join_date, getdate()) >= 2;
```

100 %

Results | Messages

	emp_id	emp_name	rating
1	101	Amit Sharma	4.5
2	102	Priya Nair	4.8
3	103	Rahul Verma	4.4
4	104	Sneha Reddy	4.1
5	106	Neha Kapoor	4.2
6	107	Vikram Rao	4.7
7	109	Karan Singh	4.3
8	111	Rohit Gupta	4.9
9	112	Divya Menon	4.4
10	113	Manish Ku...	4.6
11	114	Shreya Patel	4.2
12	116	Nikita Shah	4.5
13	117	Sanjay Mishra	4.6

4.performance trend over years

The query returns the average rating per year in order to find trend over the years.

Query

```
select
   review_year,
   AVG(rating) AS avg_rating
from Performance_Reviews
group by review_year
order by review_year;
```

100 %

Results | Messages

	review_year	avg_rating
1	2017	3.800000
2	2018	3.750000
3	2023	4.376595

Salary Analytics

5.Salary distribution per department

This query returns the average salary per department to show the salary distribution per department.

Query

```
select d.dept_id,d.dept_name,avg(s.salary) as Average_Salary
from Departments d join
employees e
on e.dept_id=d.dept_id
join salaries s
on s.emp_id=s.emp_id
group by d.dept_id,d.dept_name
order by Average_Salary;
```

100 %

Results | Messages

	dept_id	dept_name	Average_Salary
1	1	IT	678000.000000
2	2	HR	678000.000000
3	3	Finance	678000.000000
4	4	Marketing	678000.000000
5	5	Operations	678000.000000

6.Highest and lowest salary per role

This query finds the highest and lowest salary for each job role and it lists all roles and sorts them by highest salary first.

Query

```
select r.role_id,r.role_name,max(s.salary) as Highest_Salary,min(s.salary) as Lowest_Salary
from Job_Roles r
join employees e
on e.role_id=r.role_id
join  Salaries s
on s.emp_id=e.emp_id
group by r.role_id,r.role_name
```

order by Highest_Salary desc;

100 %

Results | Messages

	role_id	role_name	Highest_Salary	Lowest_Salary
1	20	Senior Software Engineer	1400000.00	1400000.00
2	9	Project Manager	1300000.00	1100000.00
3	3	Manager	1200000.00	1200000.00
4	7	Team Lead	1000000.00	950000.00
5	19	Software Engineer	950000.00	750000.00
6	8	Assistant Manager	900000.00	850000.00
7	17	Business Analyst	800000.00	700000.00
8	2	Senior Analyst	750000.00	650000.00

7.Salary vs Performance Correlation

This query retrieves each employee's salary and their performance rating.It shows the data together for comparison.

Query

```
select e.emp_id,e.emp_name,s.salary,pr.rating from employees e
join Salaries s
on s.emp_id=e.emp_id
join Performance_Reviews pr
on pr.emp_id=e.emp_id
order by e.emp_id;
```

100 %

Results | Messages

	emp_id	emp_name	salary	rating
1	101	Amit Sharma	500000.00	4.5
2	102	Priya Nair	750000.00	4.8
3	103	Rahul Ver...	550000.00	4.4
4	104	Sneha Re...	1200000...	4.1
5	105	Arjun Mehta	450000.00	4.0
6	106	Neha Kap...	420000.00	4.2
7	107	Vikram Rao	900000.00	4.7
8	108	Anjali Iyer	500000.00	3.8
9	109	Karan Singh	400000.00	4.3
10	110	Pooja Das	350000.00	4.0

8.Employees below department average salary

This query returns the employee's details whose salary is below the average salary of their department.

Query

```
select e.emp_id,e.emp_name,d.dept_name,s.salary from employees e
join Departments d
on e.dept_id=d.dept_id
join salaries s
on s.emp_id=e.emp_id
where s.salary<(select avg(s2.salary) from employees e2
join Salaries s2
on e2.emp_id =s2.emp_id
where e2.dept_id=e.dept_id);
```

100 %

Results | Messages

	emp_id	emp_name	dept_name	salary
1	101	Amit Sharma	IT	500000.00
2	102	Priya Nair	IT	750000.00
3	116	Nikita Shah	IT	600000.00
4	121	Ritesh Jain	IT	750000.00
5	126	Ananya Das	IT	700000.00
6	131	Mohit Agar...	IT	550000.00
7	136	Simran Kaur	IT	700000.00
8	146	Preeti Sha...	IT	650000.00
9	140	Aarti Desai	HR	380000.00
10	150	Lavanya ...	HR	400000.00

Department Growth

9.Hiring Trend Per year

This query shows the number of employees hired each year.It groups the employees by joining year to display the hiring trend over time.

Query

```
select year(join_date) as Year,count(*) as 'No of Employees Hired' from employees
group by YEAR(join_date);
```

100 %

Results | Messages

	Year	No of Employees Hired
1	2015	3
2	2016	6
3	2017	6
4	2018	6
5	2019	8
6	2020	7
7	2021	6
8	2022	6
9	2023	2

10.HeadCount growth by department

This query shows the number of employees in each department for every year.It helps track headcount growth over time by department.

Query

```
select d.dept_id,d.dept_name ,year(e.join_date) as 'Year',count(e.emp_id) as HeadCount from Departments d
join employees e
on e.dept_id=d.dept_id
group by d.dept_id,d.dept_name,year(e.join_date)
order by d.dept_id,d.dept_name,year(e.join_date),HeadCount;
```

100 %

Results | Messages

	dept_id	dept_name	Year	HeadCount
1	1	IT	2015	1
2	1	IT	2017	2
3	1	IT	2018	3
4	1	IT	2019	3
5	1	IT	2021	2
6	2	HR	2016	2
7	2	HR	2017	2
8	2	HR	2020	2
9	2	HR	2021	1
10	2	HR	2022	3
11	3	Finance	2015	1

Strategic Insights

11.Employees overdue for promotion

This query lists employees who have high performance ratings (greater than 4).These employees are considered overdue for promotion, and it shows them sorted by rating.

Query

```
select e.emp_id,e.emp_name,pr.rating from employees e join Performance_Reviews pr
on e.emp_id=pr.emp_id
where pr.rating>4
order by pr.rating desc;
```

100 %

Results | Messages

	emp_id	emp_name	rating
1	111	Rohit Gupta	4.9
2	143	Ashish Kapoor	4.9
3	133	Yash Thakur	4.8
4	102	Priya Nair	4.8
5	107	Vikram Rao	4.7
6	128	Riya Sharma	4.7
7	141	Varun Nair	4.7
8	138	Bhavna Joshi	4.6
9	127	Suresh Babu	4.6
10	113	Manish Kumar	4.6

12.High Salary but low performance cases

This query finds employees who have a salary above the overall average but a performance rating below 4.It helps identify high-paid employees with low performance, and results are sorted by salary and rating.

Query

```
select e.emp_id,e.emp_name,s.salary,pr.rating from employees e
join salaries s
on e.emp_id=s.emp_id
join Performance_Reviews pr
on e.emp_id=pr.emp_id
```

```
where s.salary>(select avg(salary) from salaries)
and pr.rating<4
order by s.salary desc,pr.rating desc;
```

100 %

Results | Messages

	emp_id	emp_name	salary	rating
1	115	Aditya Joshi	850000.00	3.7
2	137	Tarun Verma	750000.00	3.8

13.Location-wise salary comparison

This query returns the location wise salary distribution.

Query

```
select l.location_id,l.locatio_name,sum(s.salary) as Salary from Locations l
join employees e
on e.location_id=l.location_id
join Salaries s
on s.emp_id=e.emp_id
group by l.location_id,l.locatio_name
order by Salary desc;
```

100 %

Results | Messages

	location_id	locatio_name	Salary
1	7	Kolkata	2450000.00
2	9	Jaipur	2350000.00
3	3	Hyderabad	2300000.00
4	5	Mumbai	2230000.00
5	1	Bangalore	2200000.00
6	4	Chennai	2100000.00
7	11	Coimbatore	1950000.00
8	2	Pune	1920000.00
9	13	Bhopal	1900000.00
10	6	Delhi	1770000.00
11	17	Surat	1750000.00

14.Gender diversity ratio

This query calculates the number and percentage of employees by gender.It helps measure the gender diversity ratio within the organization.

Query

```
select gender,count(*) as total,count(*)*100/(select count(*) from employees) as percentage
from employees group by gender;
```

100 %

Results | Messages

	gender	total	percentage
1	Female	25	50
2	Male	25	50

15.Median Salary calculation

This query calculates the median salary of all employees.
It ranks salaries in order, identifies the middle one or two salaries, and takes their average to get the median.

Query

```
with ranked as(
select salary,
 ROW_NUMBER() over(order by salary) as row_num,
 count(*) over() as total
 from Salaries
)
select  avg(salary) as AverageSalary
from ranked
where row_num in ((total + 1)/2, (total + 2)/2);
```

100 %

Results | Messages

	AverageSalary
1	600000.000000

16.Identify leadership pipeline candidates

This query identifies employees who are high performers (rating > 4.5) and have at least 10 years of experience.

Query

```
select e.emp_id,e.emp_name,pr.rating,datediff(year, e.join_date, getdate()) as Experience from employees e
join Performance_Reviews pr
on e.emp_id =pr.emp_id
where pr.rating>4.5 and
datediff(year, e.join_date, getdate()) >= 10
order by Experience desc;
```

100 %

Results | Messages

	emp_id	emp_name	rating	Experience
1	111	Rohit Gupta	4.9	11
2	127	Suresh Babu	4.6	11
3	143	Ashish Kapoor	4.9	11
4	149	Prakash Naidu	4.6	10
5	117	Sanjay Mishra	4.6	10